

GAP_AUDIT_CONSOLIDATED

- [Helix Cluster OS — Consolidated Gap Audit](#)
 - [Table of Contents](#)
- [Executive Gap Summary](#)
 - [Key Metrics](#)
- [Gap 1: Dead Code / Orphaned Features](#)
 - [1.1 Overview](#)
 - [1.2 Complete Enumeration of Orphaned Packages](#)
 - [Tier 1: Control-Plane Spine \(Highest Priority for Wire-In\)](#)
 - [Tier 2: Scheduling & Resource Management](#)
 - [Tier 3: Marketplace & Economics](#)
 - [Tier 4: Federation & Data-Plane](#)
 - [Tier 5: Security Extensions](#)
 - [Tier 6: GPU & Compute](#)
 - [Tier 7: Infrastructure & Operations](#)
 - [Tier 8: Caching & Performance](#)
 - [Tier 9: LLM & Inference](#)
 - [Tier 10: Edge Computing](#)
 - [Tier 11: Testing Infrastructure](#)
 - [Tier 12: Utility & Specialized](#)
 - [1.3 Wiring Priority Order](#)
- [Gap 2: Concurrency Correctness](#)
 - [2.1 Confirmed Hazards \(F1-F10\)](#)
 - [F1: Double-Close Panic in SWIM Protocol](#)
 - [F2: Data Race on Member.State](#)
 - [F3: Untracked Goroutine in probeRandomMember](#)
 - [F4: Lock-Order Fragility in Failure Detector](#)
 - [F5: PTY Double-Close / Use-After-Close](#)
 - [F6: Unbounded Scheduler Placements Map](#)
 - [F7: Unbounded Goroutines in EventBus](#)
 - [F8: Unbuffered Delivery Stall in NATS](#)
 - [F9: No Size Cap on TieredCache Hot Tier](#)
 - [F10: Per-Event Timer Allocation](#)
 - [2.2 Additional Potential Hazards \(Not Yet Confirmed\)](#)
- [Gap 3: Critical Service Disconnections](#)
 - [CRIT-1: helix-security Stub Tokens](#)
 - [CRIT-2: helix-health HTTP-Only](#)
 - [CRIT-3: helix-build Wrong Imports](#)
 - [ARCH-1: Two-Tier Service Pattern](#)
 - [ARCH-3: Gateway Routing Incomplete](#)
- [Gap 4: Test-Type Depth](#)
 - [4.1 Test Inventory by Type](#)
 - [4.2 Missing Test Types — Detailed](#)
 - [Fuzz Tests \(Missing: 30+ targets\)](#)
 - [Integration + Stress + Chaos for Consensus](#)
 - [HelixQA Framework Untested](#)
 - [Mutation Test Coverage](#)

- [Frontend Tests \(Zero Coverage\)](#)
- [Gap 5: Security Scanning](#)
 - [5.1 Current State](#)
 - [5.2 Missing Security Scanning](#)
 - [gosec Not Promoted to Gate](#)
 - [Snyk/SonarQube Absent](#)
 - [SBOM Not Wired into Release](#)
- [Gap 6: CI/CD Pipeline](#)
 - [6.1 Current State](#)
 - [6.2 Constitution Blocks CI Re-enable](#)
 - [6.3 Local-Only Enforcement](#)
 - [6.4 PRR at 82.5%](#)
- [Anti-Bluff Package Audit](#)
 - [Audit Methodology](#)
 - [Package Audit Results](#)
 - [1. pkg/config — HIGH Risk](#)
 - [2. pkg/grpcutil — HIGH Risk](#)
 - [3. pkg/infra — HIGH Risk](#)
 - [4. pkg/jwt — HIGH Risk](#)
 - [5. pkg/leader — HIGH Risk](#)
 - [6. pkg/middleware — HIGH Risk](#)
 - [7. pkg/tracing — HIGH Risk](#)
 - [8. pkg/websocket — HIGH Risk](#)
 - [9. pkg/crypto — HIGH Risk](#)
 - [10. pkg/backoff — MEDIUM Risk](#)
 - [11. pkg/events — MEDIUM Risk](#)
 - [12. pkg/ratelimit — MEDIUM Risk](#)
 - [13. pkg/session — LOW Risk](#)
- [Phase-Specific Gap Audits](#)
 - [Phase 4: ~30% Complete, 4 P0 Blockers](#)
 - [P0 Blockers:](#)
 - [Completion Assessment:](#)
 - [Phase 7: ~18% Complete, 7/7 P0 MISSING](#)
 - [P0 Blockers \(ALL MISSING\):](#)
 - [Completion Assessment:](#)
 - [Phase 8: ~12% Complete, 4 P0 Blockers](#)
 - [P0 Blockers:](#)
 - [Phase 8B: ~8% Complete — “Textbook Double-Orphan”](#)
 - [MVP: ~60% Complete, 7 P0 Blockers](#)
 - [P0 Blockers:](#)
 - [MVP Completion Assessment:](#)

Helix Cluster OS — Consolidated Gap Audit

Field	Value
Document ID	GAC-001

Field	Value
Revision	1.0
Date	2026-03-04
Classification	INVESTIGATION — INTERNAL
Authors	Autonomous Gap Audit Agent
Status	FINAL
Scope	All structural, correctness, test, security, and CI/CD gaps

Table of Contents

- 1. [Executive Gap Summary](#)
- 2. [Gap 1: Dead Code / Orphaned Features](#)
- 3. [Gap 2: Concurrency Correctness](#)
- 4. [Gap 3: Critical Service Disconnections](#)
- 5. [Gap 4: Test-Type Depth](#)
- 6. [Gap 5: Security Scanning](#)
- 7. [Gap 6: CI/CD Pipeline](#)
- 8. [Anti-Bluff Package Audit \(24 Packages\)](#)
- 9. [Phase-Specific Gap Audits](#)

Executive Gap Summary

This consolidated gap audit identifies six structural gaps from the UNFINISHED_WORK_REPORT, 7/7 Phase 7 P0 blockers that are MISSING, 13/24 packages with PASS-bluff risk, 20 orphaned packages, and 4 CRITICAL cmd/ disconnections.

Gap	Severity	Scope	Autonomous?	Status
Dead Code / Orphaned Features	STRATEGIC	178/212 pkg/ packages	No (owner decision)	Open
Concurrency Correctness	HIGH	10 hazards (5 Major, 5 Minor)	Yes	In Progress
Critical Service Disconnections	CRITICAL	3 cmd/ + 2 architectural	Partially	In Progress
	HIGH		Yes	Open

Gap	Severity	Scope	Autonomous?	Status
Test-Type Depth		20 test types, most missing		
Security Scanning	MEDIUM	govulncheck only	Partially	Open
CI/CD Pipeline	MEDIUM	All workflows disabled	No (constitution)	Blocked

Key Metrics

Metric	Current	Target	Delta
Orphaned packages	178	0	-178
Concurrency hazards	10	0	-10
CRITICAL disconnections	3	0	-3
Test type coverage	~6/20	20/20	+14
Security scan tools	1/4 (govulncheck)	4/4	+3
CI workflows active	0/12	12/12	+12
PRR score	82.5%	95%	+12.5%
Anti-bluff pass rate	33% (10/30)	100%	+67%
Phase 7 P0 blockers	7/7 MISSING	0/7	-7

Gap 1: Dead Code / Orphaned Features

1.1 Overview

Of 212 pkg/ packages, approximately **178 have zero importers from any binary**. This represents ~51,400 non-test LOC of implemented but unreachable code. This is the single largest structural gap in the project.

1.2 Complete Enumeration of Orphaned Packages

Tier 1: Control-Plane Spine (Highest Priority for Wire-In)

These packages form the backbone of the distributed system and should be wired in first:

#	Package	Phase	LOC	Description	Triage	Wire Priority
1	pkg/leader	P4	~200	etcd leader election	Wire-in	P0
2	pkg/crdt	P4	~800	CRDT primitives	Wire-in	P0
3	pkg/deltacrdt	P4	~400	Delta-state CRDTs	Wire-in	P0
4	pkg/mvcc	P4	~300	Multi-version concurrency	Wire-in	P0
5	pkg/raftprofile	P4	~100	Raft profiling	Wire-in	P0
6	pkg/splitbrain	P4	~200	Split-brain detection	Wire-in	P0
7	pkg/splitbrainalert	P4	~100	Split-brain alerting	Wire-in	P0
8	pkg/stonith	P4	~400	STONITH fencing	Wire-in	P0
9	pkg/etcd	P4	~150	etcd key management	Wire-in	P0
10	pkg/backfill	P5	~200	Backfill scheduling	Wire-in	P1
11	pkg/priorityqueue	P5	~100	Priority queue	Wire-in	P1
12	pkg/nodeselector	P5	~100	Node selection	Wire-in	P1

Tier 2: Scheduling & Resource Management

#	Package	Phase	LOC	Description	Triage	Wire Priority
13	pkg/costsched	P5	~150	Cost-aware scheduling	Wire-in	P1
14	pkg/latencysched	P5	~150	Latency-aware scheduling	Wire-in	P1
15	pkg/rebalance	P5	~100	Workload rebalancing	Wire-in	P1
16	pkg/jobadmit	P5	~100	Job admission control	Wire-in	P1
17	pkg/smartrouter	P5	~100	Smart routing	Wire-in	P1
18	pkg/workloadrouter	P5	~100	Workload routing	Wire-in	P1
19	pkg/flowcontrol	P5	~100	Flow control	Wire-in	P1
20	pkg/latencyspot	P5	~50	Latency spot scheduling	Wire-in	P1

Tier 3: Marketplace & Economics

#	Package	Phase	LOC	Description	Triage	Wire Priority
21	pkg/marketplaceadapter	P6	~200	Akash marketplace adapter	Wire-in	P2
22	pkg/revenueopt	P6	~100	Revenue optimization	Wire-in	P2
23	pkg/billingfsm	P6	~200	Billing FSM	Wire-in	P2
24	pkg/metering	P6	~100	Metering	Wire-in	P2
25	pkg/costrouter	P6	~100	Cost routing	Wire-in	P2

#	Package	Phase	LOC	Description	Triage	Wire Priority
26	pkg/costtracker	P6	~100	Cost tracking	Wire-in	P2
27	pkg/tierdef	P6	~100	Tier definition	Wire-in	P2

Tier 4: Federation & Data-Plane

#	Package	Phase	LOC	Description	Triage	Wire Priority
28	pkg/federation	P6	~300	Federation management	Wire-in	P2
29	pkg/fedtrust	P6	~100	Federation trust	Wire-in	P2
30	pkg/spiffefed	P6	~100	SPIFFE federation	Wire-in	P2
31	pkg/cellmesh	P6	~100	Cell mesh networking	Wire-in	P2
32	pkg/dataplane	P6	~200	Data-plane pipeline	Wire-in	P2
33	pkg/helixnet	P6	~200	Network simulation	Document	P3
34	pkg/nattraversal	P6	~100	NAT traversal	Wire-in	P2

Tier 5: Security Extensions

#	Package	Phase	LOC	Description	Triage	Wire Priority
35	pkg/anticheat	P5	~100	Anti-cheat tokens	Wire-in	P1
36	pkg/attestadmit	P5	~100	Attestation admission	Wire-in	P1
37	pkg/gravladmit	P5	~100	GraVal admission	Wire-in	P1
38		P5	~100		Wire-in	P1

#	Package	Phase	LOC	Description	Triage	Wire Priority
	pkg/graavalverify			GraVal verification		
39	pkg/gpuattest	P5	~300	GPU attestation	Wire-in	P1
40	pkg/scan	P5	~100	Security scanning	Wire-in	P1
41	pkg/exportcontrol	P5	~50	Export control	Document	P3
42	pkg/imagepolicy	P5	~100	Image policy	Wire-in	P2
43	pkg/capability	P5	~100	Capability access control	Wire-in	P2
44	pkg/tofu	P5	~50	Trust On First Use	Wire-in	P2
45	pkg/anonymize	P5	~50	Data anonymization	Wire-in	P2
46	pkg/auditproof	P5	~50	Audit proof	Wire-in	P2

Tier 6: GPU & Compute

#	Package	Phase	LOC	Description	Triage	Wire Priority
47	pkg/gpucatalog	P5	~100	GPU catalog	Wire-in	P1
48	pkg/gpupool	P5	~100	GPU pooling	Wire-in	P1
49	pkg/gputopo	P5	~50	GPU topology	Wire-in	P1
50	pkg/computeprototo	P5	~200	FlatBuffers compute protocol	Wire-in	P1
51	pkg/wasm	P5	~200	WebAssembly runtime	Wire-in	P2

#	Package	Phase	LOC	Description	Triage	Wire Priority
52	pkg/kraft	P5	~200	Unikraft integration	Wire-in	P2
53	pkg/deviceprofile	P5	~100	Device profiling	Wire-in	P1
54	pkg/devicecatalog	P5	~50	Device catalog	Wire-in	P1
55	pkg/deviceplugin	P5	~200	Device plugin framework	Wire-in	P1
56	pkg/gpu	P5	~50	GPU management (thin wrapper)	Wire-in	P1

Tier 7: Infrastructure & Operations

#	Package	Phase	LOC	Description	Triage	Wire Priority
57	pkg/cloudspot	P6	~150	Cloud instance metadata	Wire-in	P2
58	pkg/storage	P6	~300	Storage backends (Redis, S3)	Wire-in	P2
59	pkg/gitops	P6	~200	GitOps integration	Wire-in	P2
60	pkg/grafanadash	P6	~200	Grafana dashboards	Wire-in	P2
61	pkg/watchtower	P6	~100	Watchtower monitoring	Wire-in	P2
62	pkg/watchmanager	P6	~100	Watch management	Wire-in	P2
63	pkg/heartbeatcoalescer	P6	~100	Heartbeat coalescing	Wire-in	P2

Tier 8: Caching & Performance

#	Package	Phase	LOC	Description	Triage	Wire Priority
64	pkg/tieredcache	P4	~100	Tiered caching	Wire-in	P1
65	pkg/burstcapacity	P4	~100	Burst capacity	Wire-in	P1
66	pkg/bursthysteresis	P4	~100	Burst hysteresis	Wire-in	P1
67	pkg/thermalwarm	P4	~50	Thermal warming	Wire-in	P2
68	pkg/ewmarank	P5	~50	EWM ranking	Wire-in	P2
69	pkg/ringavg	P4	~50	Ring buffer average	Wire-in	P2

Tier 9: LLM & Inference

#	Package	Phase	LOC	Description	Triage	Wire Priority
70	pkg/quantization	P6	~200	Model quantization	Wire-in	P2
71	pkg/chutes	P6	~200	Chutes AI client	Wire-in	P2
72	pkg/inferenceproxy	P6	~200	Inference proxy	Wire-in	P2
73	pkg/llmfailover	P6	~50	LLM failover	Wire-in	P2
74	pkg/modelintegrity	P6	~100	Model integrity	Wire-in	P2
75	pkg/modelretry	P6	~50	Model retry	Wire-in	P2

Tier 10: Edge Computing

#	Package	Phase	LOC	Description	Triage	Wire Priority
76	pkg/edge	P5	~200	Edge computing support	Wire-in	P1
77	pkg/edgefusion	P5	~100	Edge data fusion	Wire-in	P1
78	pkg/edgeheartbeat	P5	~200	Edge heartbeat	Wire-in	P1
79	pkg/edgeverify	P5	~50	Edge verification	Wire-in	P1

Tier 11: Testing Infrastructure

#	Package	Phase	LOC	Description	Triage	Wire Priority
80	pkg/testing/dst	P5	~200	DST engine	Wire-in	P1
81	pkg/testing/dstscale	P5	~100	DST scale	Wire-in	P1
82	pkg/testing/dstcompress	P5	~50	DST compression	Wire-in	P2
83	pkg/testing/dstworkload	P5	~100	DST workload	Wire-in	P1
84	pkg/testing/chaos	P5	~200	Chaos framework	Wire-in	P1
85	pkg/testing/evidence	P5	~100	Evidence collection	Wire-in	P1
86	pkg/testing/scenario	P5	~100	Scenario management	Wire-in	P1
87		P5	~100	Test runner		P1

#	Package	Phase	LOC	Description	Triage	Wire Priority
	pkg/testing/runner				Wire-in	
88	pkg/testing/snapshot	P5	~100	Snapshot testing	Wire-in	P2
89	pkg/testing/regression	P5	~100	Regression testing	Wire-in	P1
90	pkg/testing/device	P5	~200	Device provisioning	Wire-in	P2
91	pkg/testing/turmoil	P5	~100	Turmoil network testing	Wire-in	P2
92	pkg/testing/instance	P5	~100	Instance management	Wire-in	P2
93	pkg/testing/sessionfsm	P5	~100	Session FSM testing	Wire-in	P2
94	pkg/porcupine	P5	~100	Porcupine testing	Wire-in	P2

Tier 12: Utility & Specialized

#	Package	Phase	LOC	Description	Triage	Wire Priority
95	pkg/classads	P5	~300	ClassAd expressions	Wire-in	P1
96	pkg/offlinesync	P6	~200	Offline synchronization	Wire-in	P2
97	pkg/idempotent	P5	~100	Idempotent operations	Wire-in	P1
98	pkg/failconfirm	P5	~50	Failure confirmation	Wire-in	P2

#	Package	Phase	LOC	Description	Triage	Wire Priority
99	pkg/fallbackchain	P5	~50	Fallback chain	Wire-in	P2
100	pkg/headersanitize	P5	~50	Header sanitization	Wire-in	P2
101	pkg/fmea	P6	~100	FMEA	Wire-in	P2
102	pkg/forecast	P6	~100	Resource forecasting	Wire-in	P2
103	pkg/compliancedoc	P6	~50	Compliance documentation	Document	P3
104	pkg/helixtask	P5	~50	Task management	Wire-in	P2
105	pkg/local	P5	~50	Local execution	Document	P3
106	pkg/passthrough	P5	~50	Passthrough proxy	Document	P3
107	pkg/phase7matrix	P6	~50	Phase 7 matrix	Document	P3
108	pkg/pool	P5	~100	Resource pooling	Wire-in	P2
109	pkg/hxcregistry	P5	~200	HXC registry (SQLite)	Wire-in	P1
110	pkg/dst	P5	~50	DST framework (thin wrapper)	Wire-in	P1
111	pkg/burst	P5	~50	Burst management	Wire-in	P1
112	pkg/fiber	P5	~100	Fiber admission	Wire-in	P2
113	pkg/agentprovision	P5	~100	Agent provisioning	Wire-in	P1
114	pkg/epochresolve	P5	~50	Epoch resolution	Wire-in	P2
115	pkg/slotmigration	P5	~50	Slot migration	Wire-in	P2
116	pkg/hashslot	P5	~50		Wire-in	P1

#	Package	Phase	LOC	Description	Triage	Wire Priority
				Hash slot routing		
117	pkg/chaosexp	P5	~100	Chaos experiments	Wire-in	P1
118	pkg/stressmark	P5	~50	Stress marking	Document	P3
119	pkg/hybridtco	P6	~50	Hybrid TCO calculation	Document	P3
120	pkg/hlc	P4	~50	Hybrid logical clocks	Wire-in	P1
121	pkg/antientropy	P4	~100	Anti-entropy sync	Wire-in	P1

(Remaining 57 packages fall into similar categories — detailed triage available in the full package registry.)

1.3 Wiring Priority Order

Recommended wiring sequence:

1. **HA Spine** (P0): pkg/leader → pkg/etcd → pkg/crdt + pkg/deltacrdt → pkg/mvcc → pkg/raftprofile → pkg/splitbrain + pkg/splitbrainalert → pkg/stonith
 2. **Scheduler Helpers** (P1): pkg/backfill → pkg/priorityqueue → pkg/nodeselector → pkg/jobadmit → pkg/costsched → pkg/latencysched
 3. **Security Extensions** (P1): pkg/anticheat → pkg/attestadmit → pkg/gravaladmit → pkg/gpuattest → pkg/scan
 4. **GPU & Compute** (P1): pkg/gpucatalog → pkg/gpupool → pkg/deviceprofile → pkg/computeproto
 5. **Edge Computing** (P1): pkg/edge → pkg/edgeheartbeat → pkg/edgefusion
 6. **Marketplace & Economics** (P2): pkg/marketplaceadapter → pkg/revenueopt → pkg/billingfsm
 7. **Federation & Data-Plane** (P2): pkg/federation → pkg/fedtrust → pkg/dataplane
 8. **LLM & Inference** (P2): pkg/quantization → pkg/chutes → pkg/inferenceproxy
 9. **Testing Infrastructure** (P1): pkg/testing/dst → pkg/testing/chaos → pkg/testing/evidence
 10. **Utility** (P2-P3): Document-as-library or prune remaining
-

Gap 2: Concurrency Correctness

2.1 Confirmed Hazards (F1-F10)

F1: Double-Close Panic in SWIM Protocol

- **Location:** pkg/swim/protocol.go — Stop/Leave
- **Class:** Channel double-close
- **Severity:** Major / Confidence: High
- **Status:** IN PROGRESS
- **Impact:** Service crash on shutdown
- **Remediation:** sync.Once for channel close, WaitGroup for goroutines
- **Code fix:**

```
// Before (unsafe):  
func (p *Protocol) Stop() { close(p.done) }  
  
// After (safe):  
func (p *Protocol) Stop() {  
    p.closeOnce.Do(func() { close(p.done) })  
    p.cancel()  
    p.wg.Wait()  
}
```

F2: Data Race on Member.State

- **Location:** pkg/swim/protocol.go — HealthyMembers/Members
- **Class:** Read-write race on shared state
- **Severity:** Major / Confidence: High
- **Status:** IN PROGRESS
- **Impact:** Inconsistent membership view, potential crash
- **Remediation:** Protect State with mutex or use atomic operations
- **Code fix:**

```
type Member struct {  
    id      string  
    state   int32 // atomic access  
    // ...  
}  
  
func (m *Member) GetState() MemberState {  
    return MemberState(atomic.LoadInt32(&m.state))  
}  
  
func (m *Member) SetState(s MemberState) {
```

```
    atomic.StoreInt32(&m.state, int32(s))  
}
```

F3: Untracked Goroutine in probeRandomMember

- **Location:** pkg/swim/protocol.go
- **Class:** Goroutine leak
- **Severity:** Minor / Confidence: High
- **Status:** IN PROGRESS
- **Remediation:** Add to WaitGroup

F4: Lock-Order Fragility in Failure Detector

- **Location:** pkg/swim/failure_detector.go — Confirm()
- **Class:** Potential deadlock
- **Severity:** Major / Confidence: Medium
- **Status:** IN PROGRESS
- **Remediation:** Establish lock ordering, use `go test -race`

F5: PTY Double-Close / Use-After-Close

- **Location:** pkg/session/backends/native.go
- **Class:** Resource double-close
- **Severity:** Major / Confidence: High
- **Status:** IN PROGRESS
- **Remediation:** `sync.Once` for close, signal read goroutine

F6: Unbounded Scheduler Placements Map

- **Location:** pkg/scheduler/scheduler.go
- **Class:** Memory leak
- **Severity:** Major / Confidence: High
- **Status:** IN PROGRESS
- **Remediation:** Remove completed placements, add cleanup

F7: Unbounded Goroutines in EventBus

- **Location:** EventBus/pkg/bus/bus.go — PublishAsync()
- **Class:** Goroutine explosion
- **Severity:** Major / Confidence: High
- **Status:** IN PROGRESS
- **Remediation:** Bounded worker pool, backpressure

F8: Unbuffered Delivery Stall in NATS

- **Location:** EventBus/pkg/nats/nats.go
- **Class:** Channel stall
- **Severity:** Minor / Confidence: Medium
- **Status:** IN PROGRESS

- **Remediation:** Buffered channels, non-blocking send

F9: No Size Cap on TieredCache Hot Tier

- **Location:** pkg/tieredcache/tieredcache.go
- **Class:** Unbounded cache
- **Severity:** Minor / Confidence: High
- **Status:** QUEUED
- **Remediation:** LRU eviction, configurable max size

F10: Per-Event Timer Allocation

- **Location:** EventBus/pkg/bus/bus.go — trySend()
- **Class:** GC pressure
- **Severity:** Minor / Confidence: Medium
- **Status:** IN PROGRESS
- **Remediation:** Non-blocking send, shared timer pool

2.2 Additional Potential Hazards (Not Yet Confirmed)

ID	Location	Suspected Issue	Investigation Needed
F11	pkg/discovery/tier_registry.go	Concurrent map access without full lock coverage	go test -race
F12	pkg/events/nats_backend.go	NATS connection state race	Integration test with race detector
F13	internal/gpu/manager.go	GPU state mutation during detection	Race detector on GPU detection path
F14	pkg/health/rollup.go	Aggregation during concurrent health checks	Stress test
F15	internal/messaging/bus.go	Message delivery during bus shutdown	Shutdown ordering test

Gap 3: Critical Service Disconnections

CRIT-1: helix-security Stub Tokens

Description: The security service (`cmd/helix-security`) issues JWT tokens without cryptographic verification. The `ValidateToken` RPC accepts any token format, making the entire authentication system ineffective.

Location: `internal/security/server.go` , `pkg/jwt/package.go`

Evidence: - `pkg/jwt` only splits JWT strings on `.` — no signature verification - `ValidateToken` RPC returns success for any non-empty token - Gateway auth middleware (`internal/gateway/auth.go`) relies on this validation

Impact: Any client can authenticate as any user. This is a critical security vulnerability.

Remediation: 1. Implement real JWT verification using `github.com/golang-jwt/jwt/v5` 2. Add signing key management (from Vault or SPIFFE) 3. Enforce token expiration 4. Add claims validation (issuer, audience, subject) 5. Wire RBAC scopes into token claims

Estimated effort: 2-3 days

CRIT-2: helix-health HTTP-Only

Description: The health service (`cmd/helix-health`) serves health checks only via HTTP, without gRPC integration to the gateway. This means the gateway cannot use gRPC-based health checking for backend services.

Location: `internal/health/server.go`

Evidence: - Health server implements HTTP `/health` endpoint only - gRPC health service not connected to gateway routing - Aggregation collects from HTTP endpoints only

Impact: Gateway cannot make informed routing decisions based on gRPC health checks. Service mesh integration is impossible.

Remediation: 1. Implement gRPC HealthService server alongside HTTP 2. Register gRPC health service with gateway 3. Add Watch streaming for real-time health updates 4. Connect health event stream to event bus

Estimated effort: 1-2 days

CRIT-3: helix-build Wrong Imports

Description: The build service (`cmd/helix-build`) uses import paths that bypass the build orchestration layer in `internal/build/` , directly accessing lower-level functions instead of going through the orchestrator.

Location: `cmd/helix-build/main.go` , `internal/build/orchestrator.go`

Evidence: - `main.go` imports build functions directly instead of through `Orchestrator` - `SimulatedBuilder` registered in production path - Build status tracking bypasses orchestrator's state machine

Impact: Builds may not be properly tracked, monitored, or cleaned up. The orchestration layer's guarantees (retry, timeout, cancellation) are bypassed.

Remediation: 1. Fix import paths to use `Orchestrator` 2. Remove `SimulatedBuilder` from production path 3. Ensure all build operations go through orchestrator 4. Add integration test verifying orchestrator is used

Estimated effort: 1 day

ARCH-1: Two-Tier Service Pattern

Description: The codebase has a two-tier service pattern where some services have full `internal/` packages (gateway, session, scheduler, etc.) while others (agent, infra, e2ee-proxy, htmux) are implemented directly in `cmd/` with no `internal/` package. This creates inconsistency in service architecture.

Impact: Services without `internal/` packages lack proper separation of concerns and are harder to test.

Remediation: Create `internal/agent` , `internal/infra` , `internal/e2ee` , `internal/htmx` packages and extract business logic.

ARCH-3: Gateway Routing Incomplete

Description: The gateway does not route to all 14 services. Several services (advisory, build, health, policy, LLM, agent, e2ee-proxy, htmux) are not accessible through the gateway.

Impact: End users cannot access these services through the unified API. Direct service access is required, violating the gateway pattern.

Remediation: Add routing for all 14 services to the gateway, with appropriate authentication and rate limiting.

Gap 4: Test-Type Depth

4.1 Test Inventory by Type

Test Type	Count	Files	Target	Gap
Unit	~530	~400	100% packages	~50 packages missing
Integration	~64	~40	All internal/ + critical pkg/	~100 packages missing
E2E	~7	~4	All user-facing features	~30 features missing
Benchmark	~10	~5	All hot paths	~100 paths missing
Chaos	~13	~4	Consensus + scheduler	~20 subsystems missing
Fuzz	~8	~4	Crypto + parsing	~30 targets missing
Hardening	~3	~2	Security + boundary	~20 packages missing
Behavior	~3	~2	All user scenarios	~50 scenarios missing
Mutation	1 script	1	All packages (per §1.1)	~200 packages missing
Stress	0	0	Sustained load	ALL missing
Soak	0	0	7-day sustained	ALL missing
Load	0	0	50K writes/sec	ALL missing
GameDay	0	0	Manual fault injection	ALL missing
DST	~2	~2	Consensus + scheduler	

Test Type	Count	Files	Target	Gap
				~10 subsystems missing
Compliance	~1	~1	Constitution inheritance	ALL missing
Cross-Platform	~5	~3	Linux + macOS + Win/WSL2	~50 features missing
GPU Backend	~3	~2	4 vendor backends	3 backends missing
HelixQA	0	0	Challenge execution	ALL missing
Regression	~1	~1	Welch's t-test	~50 packages missing
Security Scan	1 script	1	govulncheck+gosec+trivy	2 tools missing

4.2 Missing Test Types — Detailed

Fuzz Tests (Missing: 30+ targets)

Currently covered: - pkg/hybridkex/fuzz_test.go — ML-KEM-768 key exchange - pkg/x25519session/fuzz_test.go — X25519 session - pkg/doublecrypt/fuzz_test.go — Double encryption - pkg/tracing/w3c_fuzz_test.go — W3C trace context - pkg/covgate/parse_fuzz_test.go — Coverage profile parsing - pkg/session/crdt_checkpoint_fuzz_test.go — CRDT checkpoint - pkg/computeproto — FlatBuffers roundtrip

Missing fuzz targets (CRITICAL for crypto and parsing): 1. pkg/crypto — Hash and key generation 2. pkg/jwt — Token parsing and validation 3. pkg/security/tls.go — TLS configuration parsing 4. pkg/security/vault.go — Vault secret handling 5. pkg/events/avro.go — Avro wire format parsing 6. pkg/events/avro_wire.go — Avro wire deserialization 7. pkg/events/avro_event_wire.go — Event Avro wire 8. pkg/computeproto/computeproto.go — FlatBuffers parsing 9. pkg/classads/parser.go — ClassAd expression parsing 10. pkg/config/config.go — Configuration parsing 11. pkg/netutil/cidr.go — CIDR parsing 12. pkg/openapivalidate/openapivalidate.go — OpenAPI validation 13. internal/gateway/auth.go — Auth token parsing 14. internal/security/spiffe_ca.go — SPIFFE CA operations 15. pkg/hybridkex — Additional key exchange fuzzing 16. pkg/session/crdt.go — CRDT merge fuzzing 17. pkg/session/migration.go — Migration strategy fuzzing 18. pkg/wireguard/

config.go — WireGuard config parsing 19. pkg/swim/protocol.go — SWIM message parsing 20. pkg/resources/cgroup_v2.go — cgroup parsing

Integration + Stress + Chaos for Consensus

Missing: 1. pkg/multiraft — No integration test with real etcd 2. pkg/kraft — No integration test with Unikraft 3. pkg/raftprofile — No integration test with Raft consensus 4. pkg/swim — Integration test exists but no stress/chaos 5. pkg/leader — etcd election integration exists but no stress 6. pkg/splitbrain — No integration test with real cluster 7. pkg/stonith — No integration test with real fencing

HelixQA Framework Untested

Description: The HelixQA framework's own validators and vision logic are untested. Per CLAUDE-1, an untested gate cannot certify usability.

Missing tests: 1. helixqa/pkg/validators — Image/text/video/manager/types 2. helixqa/pkg/vision/core — Core vision logic 3. helixqa/pkg/vision/detection — ORB detector 4. helixqa/cmd/helixqa-concrete-runner — Challenge runner 5. challenges/runner scaffolds across modules

Mutation Test Coverage

Currently: Only 1 script (test/mutation/run_mutations.sh) and ~5 mutation test cases.

Required per Constitution §1.1: Every test must have a paired mutation test.

Missing: ~200 packages need mutation tests.

Frontend Tests (Zero Coverage)

Description: The web frontend (web/) has zero test coverage.

Files without tests: - web/src/App.tsx - web/src/pages/*.tsx (7 pages) - web/src/components/*.tsx (4 components) - web/src/layout/*.tsx (3 layout components) - web/src/api/client.ts - web/src/api/types.ts

Gap 5: Security Scanning

5.1 Current State

Tool	Status	Integration	Evidence
govulncheck	✓ Active	scripts/security/security-scan.sh	0 reachable vulns
cyclonedx-gomod	✓ Active	scripts/gen-sbom.sh	SBOMs generated
gosec	⚠ Available	Not promoted to gate	Findings not blocking
trivy	⚠ Available	.trivyignore.yaml exists	Container scanning partial
Snyk	✗ Absent	No integration	Requires SNYK_TOKEN
SonarQube	✗ Absent	Compose file exists	Requires SONAR_TOKEN

5.2 Missing Security Scanning

gosec Not Promoted to Gate

- **Current:** gosec is available but findings don't block the build
- **Required:** Promote to make security-scan gate
- **Remediation:** Add gosec to scripts/security/security-scan.sh , fail build on HIGH severity

Snyk/SonarQube Absent

- **Current:** No Snyk or SonarQube integration
- **SonarQube:** deploy/compose/security_sonarqube.yml exists but requires SONAR_TOKEN
- **Snyk:** No integration at all
- **Remediation:** Add when owner provides tokens

SBOM Not Wired into Release

- **Current:** SBOMs are generated but not included in release artifacts
 - **Remediation:** Add SBOM files to release tarball
-

Gap 6: CI/CD Pipeline

6.1 Current State

All GitHub Actions workflows are **disabled** — stored under `.github/workflows/disabled/` :

Workflow	File	Description
<code>race.yml</code>	Go race detector	Disabled
<code>format.yml</code>	Code formatting	Disabled
<code>release.yml</code>	Release pipeline	Disabled
<code>lint.yml</code>	Linting	Disabled
<code>zig-build.yml</code>	Zig compilation	Disabled
<code>vm_integration.yml</code>	VM integration tests	Disabled
<code>cc-build.yml</code>	C/C++ build	Disabled
<code>go-build.yml</code>	Go build	Disabled
<code>docker-build.yml</code>	Docker image build	Disabled
<code>docs.yml</code>	Documentation generation	Disabled
<code>dst-sim.yml</code>	DST simulation	Disabled

6.2 Constitution Blocks CI Re-enable

The project's constitution mandates a no-CI rule. This is tracked as HXC-105/HXC-1262. CI re-enablement requires explicit owner ruling (Decision D2 in the UNFINISHED_WORK_REPORT).

6.3 Local-Only Enforcement

In the absence of CI, quality gates are enforced via local `make` targets: - `make test` — Run all tests with race detector - `make lint` — Run linters - `make gate-check` — Run orphan-prevention gates - `make security-scan` — Run security scanning - `make docs-verify` — Verify documentation sync

Gap: These are not enforced automatically. Developers can skip them.

6.4 PRR at 82.5%

The Production Readiness Review scores 66/80 = 82.5%, below the constitutional 95% bar.

CI-related gaps (items 8/9/10/16/60): - Item 8: CI build/test not active - Item 9: Release pipeline not active - Item 10: Coverage gate not enforced - Item 16: Dependency scanning not continuous - Item 60: Integration test gate not enforced

Locally closeable gaps: - Item 7: Coverage gate not enforced (pkg/covgate exists, unwired) - Item 21: HelixQA challenges not executed with sink-side evidence - Item 33: mTLS e2e incomplete - Item 34: Dep-scan continuous gate / SBOM-renovate

Anti-Bluff Package Audit

Audit Methodology

For each package, the following criteria are evaluated: 1. **Real functionality** — Does the code do real work or return hardcoded values? 2. **Mutation test coverage** — Are there paired mutation tests per Constitution §1.1? 3. **PASS-bluff risk** — Can tests pass on non-functional code? 4. **Integration potential** — Can another package actually use this?

Package Audit Results

1. pkg/config — HIGH Risk

Risk Level: HIGH

PASS-bluff findings: Load() claims to “load configuration from environment or defaults” but only returns hardcoded defaults. No env parsing, no file reading.

Evidence of bluff: Function docstring says “Load configuration from environment variables or sensible defaults” but implementation is:

```
func Load() *Config {
    return &Config{
        AppName:  "helix-cluster",
        LogLevel:  "info",
        // ... all hardcoded
    }
}
```

Remediation: Implement actual env/file loading OR rename to Default().

2. pkg/grpcutil — HIGH Risk

Risk Level: HIGH

PASS-bluff findings: Both UnaryInterceptor and StreamInterceptor are no-ops that just call the handler.

Evidence of bluff:

```
func UnaryInterceptor(ctx context.Context, req interface{},
    info *grpc.UnaryServerInfo, handler
    grpc.UnaryHandler) (interface{}, error) {
    return handler(ctx, req) // no logging, no metrics, no
        auth
}
```

Remediation: Add real interceptor behavior or remove package.

3. pkg/infra — HIGH Risk

Risk Level: HIGH

PASS-bluff findings: The entire package simulates infrastructure. Boot just sets Status: "running" in a map. VMSpawn generates fake IPs.

Evidence of bluff:

```
func (o *Orchestrator) Boot(service string) error {
    o.services[service] = ServiceStatus{Status: "running"}
    return nil // no Docker, no VMs, no cloud
}
```

Remediation: Rename to infrasim or implement real backends.

4. pkg/jwt — HIGH Risk

Risk Level: HIGH

PASS-bluff findings: Parse just splits on . — no signature verification, no algorithm check.

Evidence of bluff:

```
func Parse(tokenString string) (*Token, error) {
    parts := strings.Split(tokenString, ".")
    if len(parts) != 3 { return nil, ErrInvalidToken }
    return &Token{Header: parts[0], Payload: parts[1],
        Signature: parts[2]}, nil
    // No verification at all!
}
```

Remediation: Use github.com/golang-jwt/jwt/v5 for real verification.

5. pkg/leader — HIGH Risk

Risk Level: HIGH

PASS-bluff findings: Election is just an int32 atomic flag. Not distributed.

Evidence of bluff:

```
type Election struct { state int32 }  
func (e *Election) IsLeader() bool { return  
    atomic.LoadInt32(&e.state) == 1 }
```

Remediation: Implement etcd-based election or rename to localleader. Note: etcd_election.go exists with real implementation but is not the default.

6. pkg/middleware — HIGH Risk

Risk Level: HIGH

PASS-bluff findings: LoggingMiddleware is a no-op that calls next.ServeHTTP(w, r) with no logging.

Remediation: Implement real request logging or remove.

7. pkg/tracing — HIGH Risk

Risk Level: HIGH

PASS-bluff findings: StartSpan returns hardcoded TraceID: "trace-1", SpanID: "span-1".

Note: The package has been improved with real W3C trace context (w3c.go), gRPC tracing (grpc.go), and OTEL exporter (exporter.go). The original stub package.go may be a legacy issue.

Remediation: Remove hardcoded IDs, ensure W3C/OTel path is used.

8. pkg/websocket — HIGH Risk

Risk Level: HIGH

PASS-bluff findings: Upgrade returns nil — no actual WebSocket handshake.

Remediation: Use github.com/gorilla/websocket (already in go.mod!).

9. pkg/crypto — HIGH Risk

Risk Level: HIGH

PASS-bluff findings: No mutation tests. TestHash only checks length, not actual hash value.

Remediation: Add SHA-256 test vectors, add fuzz testing.

10. pkg/backoff — MEDIUM Risk

Risk Level: MEDIUM

PASS-bluff findings: No mutation tests. Cap logic could be removed without detection.

Remediation: Add `TestDuration_Mutation` verifying cap enforcement.

11. pkg/events — MEDIUM Risk

Risk Level: MEDIUM

PASS-bluff findings: No mutation tests. `time.Sleep` in tests is flaky.

Remediation: Add mutation tests, use deterministic timing.

12. pkg/ratelimit — MEDIUM Risk

Risk Level: MEDIUM

PASS-bluff findings: No mutation tests. `time.Sleep(2s)` in tests is slow and flaky.

Remediation: Inject clock for deterministic testing, add mutation tests.

13. pkg/session — LOW Risk

Risk Level: LOW

PASS-bluff findings: Migration strategies CRIU/DMTCP return “not implemented” — but planner correctly falls back to CRDT. This is acceptable because it’s explicitly documented.

Remediation: De-register non-functional strategies from planner.

Phase-Specific Gap Audits

Phase 4: ~30% Complete, 4 P0 Blockers

Phase 4 covers: Console hardware, edge devices, testing strategy, and cross-platform protocols.

P0 Blockers:

1. **Consensus subsystem testing** — No integration/stress/chaos tests for `pkg/multiraft`, `pkg/kraft`, `pkg/raftprofile`
2. **SWIM convergence under churn** — No Jepsen-style linearizability testing (HXC-1276)
3. **Console hardware integration** — No real hardware testing for PS4/PS5/RPi
4. **Cross-platform parity** — macOS equivalents missing for Linux-specific features (CLAUDE-2)

Completion Assessment:

Sub-phase	Target	Current	Gap
Console hardware	100% detection coverage	~40%	Missing real hardware tests
Edge devices	Full SBC/Android/iOS support	~25%	Missing mobile device support
Testing strategy	All 20 test types	~6 types	Missing 14 types
Cross-platform	Linux + macOS + Win/WSL2	Linux only	macOS/Win stubs

Phase 7: ~18% Complete, 7/7 P0 MISSING

Phase 7 covers: Production operations — monitoring, scaling, load testing, and SLO compliance.

P0 Blockers (ALL MISSING):

1. **50K writes/sec SLO** — No load testing infrastructure
2. **Monitoring/metrics pipeline** — pkg/metrics not fully wired
3. **Auto-scaling** — No scaling logic implemented
4. **Alerting** — No alert rules defined
5. **SLO dashboards** — Grafana dashboards not wired
6. **Runbook automation** — No runbooks exist
7. **Incident management** — No incident workflow

Completion Assessment:

Sub-phase	Target	Current	Gap
Monitoring	Full Prometheus + Grafana	~20%	Partial metrics, no dashboards
Scaling	Auto-scale with policies	~10%	No auto-scaling
Load testing	50K writes/sec verified	0%	No load testing
SLO compliance	All SLOs met	~15%	No SLO definitions
Alerting		0%	No alerts

Sub-phase	Target	Current	Gap
	Full alert pipeline		
Runbooks	All runbooks documented	0%	No runbooks

Phase 8: ~12% Complete, 4 P0 Blockers

Phase 8 covers: Hardening — chaos engineering, fault injection, resilience testing.

P0 Blockers:

1. **Chaos testing framework** — `pkg/testing/chaos` exists but not wired
2. **7-day soak test** — No infrastructure for sustained testing
3. **Fault injection** — `pkg/chaosexp` exists but not wired
4. **Resilience testing** — No automated resilience verification

Phase 8B: ~8% Complete — “Textbook Double-Orphan”

Description: Phase 8B packages are doubly orphaned — they are not only unwired from binaries, but they also depend on other orphaned packages. This creates chains of dead code.

Affected packages: - `pkg/marketplaceadapter` → depends on `pkg/federation` (orphaned) - `pkg/fedtrust` → depends on `pkg/spiffefed` (orphaned) - `pkg/edgefusion` → depends on `pkg/edge` (orphaned) - `pkg/chaosexp` → depends on `pkg/testing/chaos` (orphaned)

Remediation: Wire from the bottom up — resolve dependencies before dependents.

MVP: ~60% Complete, 7 P0 Blockers

MVP scope: Minimum viable product — cluster management, scheduling, sessions, builds, and basic security.

P0 Blockers:

1. **CRIT-1: JWT stub tokens** — Authentication doesn't work
2. **CRIT-2: Health HTTP-only** — No gRPC health checking
3. **CRIT-3: Build wrong imports** — Build orchestration bypassed
4. **helixd is a hollow shell** — Core daemon does nothing
5. **Gateway routing incomplete** — Not all services accessible

- 6. **No real configuration management** — pkg/config returns hardcoded values
- 7. **No CI enforcement** — Quality gates not enforced

MVP Completion Assessment:

Feature	Target	Current	Gap
Cluster management	Full node lifecycle	~50%	Missing health, scaling
Scheduling	Omega-model with backfill	~40%	Missing helpers
Sessions	CRDT with migration	~80%	Missing CRIU migration
Builds	Podman with streaming	~60%	Import fix needed
Security	mTLS + RBAC + JWT	~30%	JWT stub tokens
Gateway	Full routing	~50%	Missing service routes
Configuration	Environment + file	~10%	Hardcoded defaults

End of Consolidated Gap Audit

Document Statistics: - Total gaps identified: 6 major categories - Total orphaned packages enumerated: 121+ of 178 - Total concurrency hazards: 10 confirmed + 5 potential - Total CRITICAL disconnections: 3 + 2 architectural - Total missing test types: 14 of 20 - Total phase-specific P0 blockers: 22 - Total anti-bluff package audits: 13 detailed